Secure DevOps

Setting up a secure environment for Ansible

PRESENTED BY:
Joel Kirch

© CC BY-NC-ND — This license enables reusers to copy and distribute the material in any medium or format in unadapted form only, for noncommercial purposes only, and only so long as attribution is given to the creator.

Today's Topics

- Many demos and tutorials skip over how to set up a secure environment for Ansible
  - SSH & GPG Keys, SSH-Agent, GPG-Agent
  - `pass` – The Unix Password Manager
  - Ansible Vault
- **SSH Setup Demo**
- **Ansible Setup Demo**
- **Ansible Demo**

SSH Setup Demo

Generate SSH Keys

- Generate an ED25519 SSH key with the comment “DemoSSHKey” and save it using that same filename – USE A PASSWORD to protect your SSH Key.

```
ssh-keygen -t ed25519 -C "DemoSSHKey" -f DemoSSHKey
```

- This command breaks down as follows:
  - `-t ed25519`: Specifies the type of key to generate (ED25519).
  - `-C "DemoSSHKey"`: Adds the comment “DemoSSHKey” to the key
  - `-f DemoSSHKey`: Saves the private key to a file named `DemoSSHKey` and the public key to a file named `DemoSSHKey.pub`.
- After running this command, you’ll have two files:
  - DemoSSHKey (the private key)
  - DemoSSHKey.pub (the corresponding public key)

Upload SSH Public Key

- Copy your public SSH key and upload it to your server or cloud provider
- The public key is safe to share – that's the whole point
- For cloud providers (e.g., DigitalOcean), paste the key in the SSH Keys section of the console

```
cat ~/.ssh/DemoSSHKey.pub
```

RSA 4096 vs ED25519


```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQD4kiiiQH2Avx4rzOsNf
fLvnEZAMuEF7CAOcpg3L6BO1aFu6jEFvr/NonnL1tPN/OwDoTeEMG4k7B
N3dAXFFkgFAGAT+2RNvTnFb+nhyReeEkwZg+Iw9+stfxITrtvKgoNQ76n
884P79FciMbn9QP6BmCXsTUQhMFMvWHPO3rwYXFt9f/pUgW6QCmpIw5i3f
KgQfOWGFVsjPWXMCXrNiIDDk/7iVUqXg8FKFUvQUG1DGDcspPdelOgsJSX
y14Wov4l4sSlViYqe6WQXKA6aIvsQGk+DfZErKPWMg+2OLCAYYQeD1+r4Q
iLbMHbHNK26WBmkjzv0pgkbEdBdrscRaTi8AFCWelyqsoxxz4GEKOCG7NoB
idlOKTf95OqDpmAnZcwRDTLWbYlLBxgL8+Pmjgku9Pnzhf2qipQs94A+te
EgZiPlpIXSitqcBmJOaamCTMrYE9BkylM7hvGkfx7QZYWktQkgdPKKl8ac
KDM77nc1SWc1vQ2pHhq53xk08cNWrOMZaxqZaDMGncxp83GnOEjkZhr6Xw
9T5/ Demo_SSH_RSA_Key
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDGwqTc2eyAVxspuLyNERj
9ktEHdkRwdvKpAaO8ZP68 Demo_SSH_ED25519_Key
```

Configure SSH Client

```
Host ansible
    HostName 192.168.122.100
    User root
    IdentityFile ~/.ssh/DemoSSHKey
```

`Host ansible`: Creates an alias “ansible” for easy connection.

`HostName 192.168.122.100`: Sets the server’s IP address.

`User root`: Uses “root” as the default username.

`IdentityFile ~/.ssh/DemoSSHKey`: Specifies the SSH key for secure login.

SSH to ansible, part 1

```
$ ssh ansible
Enter passphrase for key '~/.ssh/DemoSSHKey':
Welcome to Ubuntu 24.04 LTS

 System load: 0.08 Processes: 104
 Usage of /: 19.2% Users logged in: 0
 Memory usage: 36% IPv4: 192.168.122.100

root@ansible:~#
```

- Notice that I have to enter a password to use my SSH key.
- Let's fix that by using SSH agent
- Add this line to your config file:
  `AddKeysToAgent yes`

$\Rightarrow$

```
Host ansible
    HostName 192.168.122.100
    User root
    AddKeysToAgent yes
    IdentityFile ~/.ssh/DemoSSHKey
```

CyberSecurity Time

- Update the `sshd_config` on the ansible server
  - Disable root login: `PermitRootLogin no` ensures that root cannot access the server via SSH.
  - Enforce public key authentication: `PubkeyAuthentication yes` and `PasswordAuthentication no` require that all users authenticate with an SSH key.
  - Do not forget to restart SSHD: `systemctl restart ssh.service`

```
# Authentication:

#LoginGraceTime 2m
PermitRootLogin no <--
#StrictModes yes
#MaxAuthTries 6

PubkeyAuthentication yes <--

...

PasswordAuthentication no <--
#PermitEmptyPasswords no
```

Update ssh config

- Update my ssh client config to use `ansible_user` instead of `root`

```
Host ansible
    HostName 192.168.122.100
    User ansible_user
    AddKeysToAgent yes
    IdentityFile ~/.ssh/DemoSSHKey
```

```
$ ssh ansible
root@192.168.122.100: Permission denied
  (publickey).

$ vim ~/.ssh/config

$ ssh ansible
Welcome to Ubuntu 24.04 LTS
...
ansible_user@ansible:~$
```

SSH Setup Summary

- Created SSH keys, uploaded the public key to cloud provider
- Configured our client to use SSH Agent with SSH keys
- Connected to our ansible server, using SSH keys
- Hardened SSHd on the ansible server
- Verified that our settings worked
- **QUESTION: Are we doing DevOps?**

Ansible Setup Demo

Install Ansible

```
ansible_user@ansible:~$ ./install_ansible.sh
Reading package lists... Done
Building dependency tree... Done
...
The following NEW packages will be installed:
  python3 python3-pip python3-venv ...
...
Do you want to continue? [Y/n]
```

- Use `git` to download the install script
- `chmod +x install_ansible.sh`
- `./install_ansible.sh`

Logout & Login

```
ansible_user@ansible:~$ ./install_ansible.sh
...
~/.local/bin has been added to PATH, but you need to open
a new terminal or re-login for this PATH change to take effect.

ansible_user@ansible:~$ ansible --version
ansible [core 2.18.1]
  config file = None
  ...
  python version = 3.12.3
```

Ansible Installed

```
ansible_user@ansible:~$ ansible --version
ansible [core 2.18.1]
  config file = None
  configured module search path =
    ['/home/ansible_user/.ansible/plugins/modules',
     '/usr/share/ansible/plugins/modules']
  ansible python module location =
    /home/ansible_user/.local/share/uv/...
  executable location =
    /home/ansible_user/.local/bin/ansible
  python version = 3.12.3
  jinja version = 3.1.4
  libyaml = True
```

Ansible Demo

Our First Ansible Playbook

- Create an inventory file
- Write a playbook
- Run the playbook

```
[local]
localhost ansible_connection=local
```

```
---
- name: Ping localhost
  hosts: local
  tasks:
    - name: Ping the localhost
      ping:
```

```
ansible_user@ansible:~/secure_devops_demo$ ansible-playbook ping.yml

PLAY [Ping localhost] ***************************************************
TASK [Gathering Facts] ***************************************************
ok: [localhost]
TASK [Ping the localhost] ************************************************
ok: [localhost]
PLAY RECAP ***************************************************************
localhost : ok=2 changed=0 unreachable=0 failed=0
```

ansible.cfg

```
[defaults]
inventory = inventory.ini
command_warnings = False
deprecation_warnings = False
interpreter_python =
    /usr/bin/python3
```

Add the inventory location and turn off those warnings

Now you don't have to pass the `-i inventory.ini` each time you run a playbook

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook ping.yml

PLAY [Ping localhost] *****************
TASK [Gathering Facts] ****************
ok: [localhost]
TASK [Ping the localhost] *************
ok: [localhost]
PLAY RECAP ****************************
localhost : ok=2 changed=0 failed=0
```

Update ansible with Ansible

```
---
- name: Update and upgrade packages
    on localhost
  hosts: local
  become: yes tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
    - name: Upgrade all packages
      apt:
        upgrade: dist
```

- Notice that we are using `become` to use sudo
- It failed?
- Why?

```
ansible_user@ansible:~/secure_devops_demo$ ansible-playbook update.yml
PLAY [Update and upgrade packages on localhost] **************************
TASK [Gathering Facts] ***************************************************
fatal: [localhost]: FAILED! => {"msg": "sudo: a password is required\n"}
```

Ansible with sudo

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook update.yml --ask-become-pass
BECOME password:

PLAY [Update and upgrade packages on localhost] ********
TASK [Gathering Facts] *********************************
ok: [localhost]
TASK [Update apt cache] ********************************
changed: [localhost]
```

- It worked!
- Typing passwords each time you run a playbook is no good.
- Let's fix that.

Storing Secrets in Ansible

- Ansible can store secrets in a “vault”
  - Uses AES 256 to encrypt the data
  - Needs to be unlocked before using
- You could store the `ansible_user` password for sudo in the vault and run the playbook:
  `ansible-playbook update.yml -ask-vault-pass`
- **Not much better than just running:**
  `ansible-playbook update.yml -ask-become-pass`

Ansible Vault

```
ansible_user@ansible:~/secure_devops_demo$ ansible-vault view vault.yml
Vault password:
ansible_become_password: ************
```

```
ansible_user@ansible:~/secure_devops_demo$ cat vault.yml
$ANSIBLE_VAULT;1.1;AES256
6136616539613930613666366663430656266383433333662643930636431 30
3366366336363336363432336131343539383632313330393334363133383 0
3630653331363339313634343631363433353633353333333636439393636 36
3866396230363336393930613338363133383633373833333036363337363 1
```

Let's use pass

```
ansible_user@ansible:~/secure_devops_demo$ ./install_setup_pass.sh
Installing pass, the Unix password manager...
[sudo] password for ansible_user:

gpg: directory '/home/ansible_user/.gnupg' created
gpg: keybox '/home/ansible_user/.gnupg/pubring.kbx' created
Generating a new GPG key for pass...
...
Password store initialized for 2466552D8747919
Pass initialized with GPG key ID: 2466552D8747919
Enter the password for the Ansible Vault:

Enter contents of ansible/vault_password and press Ctrl+D when finished:

Ansible Vault password has been securely stored in pass under
  'ansible/vault_password'.
ansible_user@ansible:~/secure_devops_demo$ pass ansible/vault_password
```

pass uses GPG

pass uses GPG to encrypt/decrypt passwords

```
ansible_user@ansible:~$ tree
  ~/.password-store/
/home/ansible_user/.password-store/
  ansible
    vault_password.gpg
```

pass just stores each entry as its own GPG encrypted file

GPG uses an agent, just like the SSH Agent

```
/run/user/1000/gnupg/S.gpg-agent

SSH Agent PID:
25296 ssh-agent
25299 ssh-agent
```

```
ansible_user@ansible:~$ file
  ~/.password-store/ansible/
    vault_password.gpg
...PGP RSA encrypted session key
  - keyid: DEF15331 A3CB758D
  RSA (Encrypt or Sign) 2048b

ansible_user@ansible:~$ xxd
  ~/.password-store/ansible/
    vault_password.gpg
00000000: 8501 0c03 def1 5331
00000010: 8ee9 9d32 2e5a 0c68
00000020: bf34 3e73 2e26 32b6
...
```

Update `ansible.cfg` for `pass`

- Update the ansible.cfg to call a script to use pass to unlock our vault

```
[defaults]
inventory = inventory.ini
command_warnings = False
deprecation_warnings = False
interpreter_python = /usr/bin/python3
vault_password_file =
    bin/get_vault_pass.sh
```

```
#!/bin/bash
pass ansible/vault_password
```

- Wait, what were we doing again?
- Updating the packages on the ansible server, securely using NO PLAINTEXT passwords.

Update our `update.yml`

```
---
- name: Update and upgrade packages
    on localhost
  hosts: local
  become: yes
  vars_files: - vault.yml tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
    - name: Upgrade all packages
      apt:
        upgrade: dist
```

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook update.yml

PLAY [Update and upgrade packages
  on localhost] ********************

TASK [Gathering Facts] *************
ok: [localhost]

TASK [Update apt cache] ************
changed: [localhost]

TASK [Upgrade all packages] ********
ok: [localhost]

PLAY RECAP *************************
localhost : ok=3 changed=1
  unreachable=0 failed=0
```

Okay...

- Couldn't I have just run:

```
apt update && apt upgrade -y
```

- **Yes, but that's not DevOps**
  - It doesn't scale to dozens / hundreds / thousands of machines
  - Error prone
  - You don't get "infrastructure as code"

Let's spin up more machines

- We are going to need our SSH Keys for this
- 3 local VMs on QEMU/KVM – no internet needed

```
# SSH configuration for servers
Host server1
    HostName 192.168.122.101
    User ansible_user
    IdentityFile ~/.ssh/DemoSSHKey
    AddKeysToAgent yes

Host server2
    HostName 192.168.122.102
    User ansible_user
    IdentityFile ~/.ssh/DemoSSHKey
    AddKeysToAgent yes

Host server3
    HostName 192.168.122.103
    User ansible_user
    IdentityFile ~/.ssh/DemoSSHKey
    AddKeysToAgent yes
```

- Update 2 files:
  - `~/.ssh/config`
  - `inventory.ini`

```
[local]
localhost ansible_connection=local

[servers]
192.168.122.101
192.168.122.102
192.168.122.103
```

Ansible to update `.bashrc`

```
---
- hosts: localhost
  tasks:
    - name: Ensure ssh-agent is started
        and environment variables are saved
      blockinfile:
        path: ~/.bashrc
        marker: "# {mark} Ansible SSH
          agent configuration"
        block: |
          # Ansible modified: SSH agent setup
          if [ -z "$SSH_AUTH_SOCK" ]; then
            eval "$(ssh-agent -s)" >
              ~/.ssh-agent-variables
            ssh-add ~/.ssh/DemoSSHKey
          fi
          if [ -f ~/.ssh-agent-variables ]
          then
            source ~/.ssh-agent-variables
          fi
        create: yes
```

Now the `.bashrc` file is updated to start the SSH-Agent when the user logs in

```
# BEGIN Ansible SSH agent
# configuration
# Ansible modified: SSH agent
# setup
if [ -z "$SSH_AUTH_SOCK" ]; then
    eval "$(ssh-agent -s)" >
      ~/.ssh-agent-variables
    ssh-add ~/.ssh/DemoSSHKey
fi
if [ -f ~/.ssh-agent-variables ]
then
    source ~/.ssh-agent-variables
fi
# END Ansible SSH agent
# configuration
```

Idempotent

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook add_ssh_key.yml

PLAY [localhost] ***********************
TASK [Gathering Facts] *****************
ok: [localhost]
TASK [Ensure ssh-agent is started...] **
changed: [localhost]

PLAY RECAP *****************************
localhost : ok=2 changed=1

ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook add_ssh_key.yml

PLAY [localhost] ***********************
TASK [Gathering Facts] *****************
ok: [localhost]
TASK [Ensure ssh-agent is started...] **
ok: [localhost]

PLAY RECAP *****************************
localhost : ok=2 changed=0
```

- Idempotency ensures repeatability: Running the same operation multiple times produces the same outcome without unintended side effects.
- Prevents duplication and redundancy: Ensures that changes are only applied if necessary, making configuration management predictable and reliable.

Now we are ready

- Oh, right. First time SSHing, getting the fingerprint message.
- Also, we are trying to connect as `ansible_user`

```
ansible_user@ansible:~/secure_devops_demo$ ansible-playbook ping-servers.yml

PLAY [Ping servers] ****************************************************
TASK [Gathering Facts] *************************************************
fatal: [192.168.122.101]: UNREACHABLE! => {"changed": false, "msg": "Failed
  to connect to the host via ssh: ansible_user@192.168.122.101: Permission
  denied (publickey).", "unreachable": true}
The authenticity of host '192.168.122.102 (192.168.122.102)' can't be
  established.
ED25519 key fingerprint is SHA256:PzXwrkP3W+...
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
fatal: [192.168.122.102]: UNREACHABLE! => {"changed": false, "msg": "Failed
  to connect to the host via ssh: Permission denied (publickey)."}
```

Add `ansible_user` to servers

```yaml
---
- name: Add ansible_user to the sudo
    group, set up SSH key, and set
    password
  hosts: servers
  become: true
  remote_user: root vars_files:
    - vault.yml
  tasks:
    - name: Install passlib on remote
        hosts (if required)
      ansible.builtin.package:
        name: python3-passlib
        state: present
    - name: Ensure ansible_user is in
        the sudo group with a hashed
        password
      user:
        name: ansible_user
        groups: sudo
        append: true
        password: "{{ ansible_user_password
          | password_hash('sha512') }}"
```

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook add_ansible_user.yml

PLAY [Add ansible_user to the sudo
  group, set up SSH key, and set
  password] ************************

TASK [Gathering Facts] *************
ok: [192.168.122.101]
ok: [192.168.122.103]
ok: [192.168.122.102]

TASK [Install passlib...] **********
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]

TASK [Ensure ansible_user...] ******
changed: [192.168.122.101]
changed: [192.168.122.102]
changed: [192.168.122.103]

TASK [Create .ssh directory...] ****
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]
```

Verify `ansible_user` can connect first

```
ansible_user@ansible:~/secure_devops_demo$ ansible-playbook ping-servers.yml

PLAY [Ping servers] ***************************************************

TASK [Gathering Facts] ************************************************
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]

TASK [Ping the localhost] *********************************************
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]

PLAY RECAP ************************************************************
192.168.122.101 : ok=2 changed=0 unreachable=0 failed=0
192.168.122.102 : ok=2 changed=0 unreachable=0 failed=0
192.168.122.103 : ok=2 changed=0 unreachable=0 failed=0
```

Verify before you lock the door.

SSHD Hardening

```
---
- name: Configure SSHD for Ubuntu
    22.04, 24.04, and 24.10
  hosts: servers
  become: true
  vars_files: - vault.yml tasks:
    - name: Ensure root login is
        disabled
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: 'PermitRootLogin no'
        state: present
    - name: Ensure public key
        authentication is enabled
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PubkeyAuthentication'
        line: 'PubkeyAuthentication yes'
        state: present
    - name: Disable password
        authentication
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication no'
        state: present
```

```
ansible_user@ansible:~/secure_devops_demo$
  ansible-playbook
    sshd_hardening-servers.yml

PLAY [Configure SSHD for Ubuntu
  22.04, 24.04, and 24.10] *********

TASK [Gathering Facts] *************
ok: [192.168.122.101]
ok: [192.168.122.103]
ok: [192.168.122.102]

TASK [Ensure root login is
  disabled] **************************
changed: [192.168.122.101]
changed: [192.168.122.102]
changed: [192.168.122.103]

TASK [Ensure public key auth...] ***
changed: [192.168.122.101]
changed: [192.168.122.102]
changed: [192.168.122.103]

TASK [Disable password auth...] ****
changed: [192.168.122.101]
changed: [192.168.122.102]
changed: [192.168.122.103]
```

Now we can ping servers

```
ansible_user@ansible:~/secure_devops_demo$ ansible-playbook ping-servers.yml

PLAY [Ping servers] ***************************************************

TASK [Gathering Facts] ************************************************
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]

TASK [Ping the localhost] *********************************************
ok: [192.168.122.101]
ok: [192.168.122.102]
ok: [192.168.122.103]

PLAY RECAP ************************************************************
192.168.122.101 : ok=2 changed=0 unreachable=0 failed=0
192.168.122.102 : ok=2 changed=0 unreachable=0 failed=0
192.168.122.103 : ok=2 changed=0 unreachable=0 failed=0
```

Ad-hoc commands

```
ansible servers -m shell -a "df -h"
```

```
192.168.122.101 | CHANGED | rc=0 >>
Filesystem Size Used Avail Use% Mounted
/dev/vda1 9.6G 1.7G 7.9G 18% /
tmpfs 46M 1008K 45M 3% /run
...
192.168.122.102 | CHANGED | rc=0 >>
Filesystem Size Used Avail Use% Mounted
/dev/vda1 9.6G 1.7G 7.9G 18% /
...
192.168.122.103 | CHANGED | rc=0 >>
Filesystem Size Used Avail Use% Mounted
/dev/vda1 9.6G 1.7G 7.9G 18% /
```

```
ansible servers -a "ss -tuln"
```

```
192.168.122.101 | CHANGED | rc=0 >>
Netid State Recv-Q Send-Q
  Local Address:Port
udp UNCONN 0 0
  127.0.0.53%lo:53
tcp LISTEN 0 4096
  127.0.0.53%lo:53
tcp LISTEN 0 128
  0.0.0.0:22
tcp LISTEN 0 128
  [::]:22
192.168.122.102 | CHANGED | rc=0 >>
...
192.168.122.103 | CHANGED | rc=0 >>
...
```

Summary

- DevOps is awesome!
- Many demos and tutorials skip over how to set up a secure environment for Ansible
  - SSH & GPG Keys
  - SSH-Agent
  - GPG-Agent
  - `pass` – The Unix Password Manager
  - Ansible Vault

Questions?

CONTACT:
jkirch (at) tcc dot edu

© CC BY-NC-ND — This license enables reusers to copy and distribute the material in any medium or format in unadapted form only, for noncommercial purposes only, and only so long as attribution is given to the creator.